Listing 4.10 Initial mount information for the ChildComponent

```
id: ChildComponent
    phase: mount
    actualDuration: 0.19999992847442627
    baseDuration: 0
    startTime: 132.20000004768372
    commitTime: 132.70000004768372
```

Listing 4.11 shows update information for `ChildComponent` after incrementing the `count`.

Listing 4.11 Update information for ChildComponent after incrementing count

```
id: ChildComponent
    phase: update
    actualDuration: 0.10000002384185791
    baseDuration: 0.10000002384185791
    startTime: 24959.100000023842
    commitTime: 24959.600000023842
```

By analyzing these metrics, developers can pinpoint performance bottlenecks —such as components with long `actualDuration` times or unexpected re-renders—and take targeted action to optimize rendering efficiency.

Additionally, there are several other effective ways to check if a React component is re-rendering:

- *Console logging*—Strategically placing `console.log` statements inside a component's render method offers a simple yet effective way to track re-renders. This approach is particularly useful for debugging and understanding the component's lifecycle.
- `useEffect` *hook (mount Only)*—Using the `useEffect` hook with an empty dependency array (`[]`) enables logging a message precisely once when the component initially mounts. This is helpful for verifying initial setup or state.
- `useEffect` *hook with dependencies*—When used with a dependency array, the `useEffect` hook becomes a powerful tool to detect changes in specific props or state variables. A `console.log` or other diagnostic action can be triggered only when those dependencies change, providing